

# CUSTOMER BEHAVIOR ANALYSIS

End-to-end project report: Python · MySQL · Power BI

An end-to-end analytics project covering data cleaning and feature engineering in Python, business analysis in SQL, and an interactive Power BI dashboard built on 3,900 retail customers.

[Shubham Khatri](#) | [GitHub](#) | [LinkedIn](#)

## 1. Project Overview

**Goal:** understand who the customers are, what they buy, how they shop and which groups drive revenue, so a retail business can target marketing, subscriptions and loyalty programs better.

| Item         | Detail                                                                                                            |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| Dataset      | 3,900 transactions, 18 raw columns (demographics, product, payment, shipping, discount, review, purchase history) |
| Pipeline     | CSV → Python (clean and engineer) → MySQL (query) → Power BI (visualise)                                          |
| Tools        | Python (Pandas, NumPy, SQLAlchemy), Jupyter, MySQL, Power BI                                                      |
| Deliverables | Jupyter notebook, 10 SQL business queries, one-page dashboard                                                     |

**3.90K**

Customers

**\$233K**

Total revenue

**\$59.76**

Avg purchase

**3.75**

Avg review (of 5)

## 2. Data Preparation in Python

Every step from the notebook `customer_analytics.ipynb`, with what the code does and why. Paste extra screenshots into the dashed boxes.

### Step 1: Load the data

Import pandas and NumPy, then read the raw CSV into a DataFrame.

```
import numpy as np
import pandas as pd
cust_df = pd.read_csv("customer_shopping_behavior.csv")
```

### Step 2: Inspect structure and quality

`info()` shows column types and non-null counts; `describe(include="all")` gives statistics for numeric and text columns. Finding: every column is complete except Review Rating (3,863 of 3,900 filled, so 37 are missing). Ages range 18 to 70; 2,652 customers are male; Clothing is the top category (1,737).

```
cust_df.info()
cust_df.describe(include="all")
```

## Output

```
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns)
Review Rating    3863 non-null   float64    <- 37 missing values
```

|        | # Customer ID      | # Age             | Gender        | Item Purchased | Category      | # Purchase Amount (USD) | Location      |
|--------|--------------------|-------------------|---------------|----------------|---------------|-------------------------|---------------|
| count  | 3900.0             | 3900.0            | 3900          | 3900           | 3900          | 3900.0                  | 3900          |
| unique | Missing value      | Missing value     | 2             | 25             | 4             | Missing value           | 50            |
| top    | Missing value      | Missing value     | Male          | Blouse         | Clothing      | Missing value           | Montana       |
| freq   | Missing value      | Missing value     | 2652          | 171            | 1737          | Missing value           | 96            |
| mean   | 1950.5             | 44.06846153846154 | Missing value | Missing value  | Missing value | 59.76435897435898       | Missing value |
| std    | 1125.9773532358456 | 15.20758912716238 | Missing value | Missing value  | Missing value | 23.685392250875307      | Missing value |
| min    | 1.0                | 18.0              | Missing value | Missing value  | Missing value | 20.0                    | Missing value |
| 25%    | 975.75             | 31.0              | Missing value | Missing value  | Missing value | 39.0                    | Missing value |
| 50%    | 1950.5             | 44.0              | Missing value | Missing value  | Missing value | 60.0                    | Missing value |
| 75%    | 2925.25            | 57.0              | Missing value | Missing value  | Missing value | 81.0                    | Missing value |

11 rows x 18 cols | 10 per page | Page 1 of 2

## Step 3: Fix missing review ratings

Missing ratings are filled with the median rating of the same product category. The median resists outliers, and grouping by category is more accurate than one global value.

```
cust_df["Review Rating"] = cust_df.groupby("Category")["Review Rating"] \
    .transform(lambda x: x.fillna(x.median()))
```

## Step 4: Standardise column names

Lower-case snake\_case names are easier to use in SQL and Power BI. purchase\_amount\_(usd) is shortened to purchase\_amount.

```
cust_df.columns = cust_df.columns.str.lower()
cust_df.columns = cust_df.columns.str.replace(" ", "_")
cust_df = cust_df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})
```

## Output

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
      'purchase_amount', 'location', 'size', 'color', 'season',
      'review_rating', 'subscription_status', 'shipping_type',
      'discount_applied', 'promo_code_used', 'previous_purchases',
      'payment_method', 'frequency_of_purchases'], dtype='object')
```

## Step 5: Feature: age groups

pd.qcut splits ages into 4 equal-sized quartile groups so each segment holds a similar number of customers.

```
age_group = ["Young Adult", "Adult", "Middle-Aged", "Senior"]
cust_df["age_group"] = pd.qcut(cust_df["age"], q=4, labels=age_group)
```

## Output

```
age  age_group
55   Middle-Aged
19   Young Adult
63   Senior
```

## Step 6: Feature: purchase frequency in days

Text such as Weekly or Quarterly cannot be averaged or sorted, so each value is mapped to a number of days between purchases.

```
frequency_mapping = {"Fortnightly":14, "Weekly":7, "Annually":365, "Quarterly":90,
                     "Bi-Weekly":14, "Monthly":30, "Every 3 Months":90}
cust_df['purchase_frequency_days'] = cust_df['frequency_of_purchases'].map(frequency_mapping)
```

|   | # purchase_frequency_days | frequency_of_purchases |
|---|---------------------------|------------------------|
| 0 | 14                        | Fortnightly            |
| 1 | 14                        | Fortnightly            |
| 2 | 7                         | Weekly                 |
| 3 | 7                         | Weekly                 |
| 4 | 365                       | Annually               |
| 5 | 7                         | Weekly                 |
| 6 | 90                        | Quarterly              |
| 7 | 7                         | Weekly                 |
| 8 | 365                       | Annually               |
| 9 | 90                        | Quarterly              |

3,900 rows x 2 cols 10 per page

## Step 7: Remove a redundant column

A check showed discount\_applied and promo\_code\_used are identical on every row, so the duplicate is dropped.

```
(cust_df['discount_applied'] == cust_df['promo_code_used']).all()
cust_df.drop("promo_code_used", axis=1, inplace=True)
```

### Output

True

## Step 8: Feature: customer segments

pd.cut buckets customers by past purchases: 0 to 1 = New, 2 to 5 = Returning, 6 or more = Loyal.

```
cust_df['customer_segment'] = pd.cut(cust_df['previous_purchases'],
                                     bins=[-1, 1, 5, float('inf')], labels=['New', 'Returning', 'Loyal'])
```

| # customer_id | # age | gender | item_purchased | category    | # purchase_amount | location      | size |
|---------------|-------|--------|----------------|-------------|-------------------|---------------|------|
| 1             | 55    | Male   | Blouse         | Clothing    | 53                | Kentucky      | L    |
| 2             | 19    | Male   | Sweater        | Clothing    | 64                | Maine         | L    |
| 3             | 50    | Male   | Jeans          | Clothing    | 73                | Massachusetts | S    |
| 4             | 21    | Male   | Sandals        | Footwear    | 90                | Rhode Island  | M    |
| 5             | 45    | Male   | Blouse         | Clothing    | 49                | Oregon        | M    |
| 6             | 46    | Male   | Sneakers       | Footwear    | 20                | Wyoming       | M    |
| 7             | 63    | Male   | Shirt          | Clothing    | 85                | Montana       | M    |
| 8             | 27    | Male   | Shorts         | Clothing    | 34                | Louisiana     | L    |
| 9             | 26    | Male   | Coat           | Outerwear   | 97                | West Virginia | L    |
| 10            | 57    | Male   | Handbag        | Accessories | 31                | Missouri      | M    |

3,900 rows x 20 cols 10 per page

<< < Page 1 of 390 > >>

## Step 9: Validate and export to MySQL

A final info() confirms 3,900 rows with no nulls across 20 columns. The cleaned data is saved to CSV and loaded into the MySQL table shop through SQLAlchemy for the SQL analysis.

```

cust_df.info()
cust_df.to_csv("customer_shopping_behavior_cleaned.csv", index=False)

from sqlalchemy import create_engine
engine = create_engine("mysql+mysqlconnector:// root:Password@localhost:3306/customer")
cust_df.to_sql("shop", con=engine, if_exists="replace", index=False)

```

#### Output

DataFrame exported to MySQL successfully!

### Summary of changes

| Change          | Result                                               |
|-----------------|------------------------------------------------------|
| Missing values  | 37 ratings imputed, 0 nulls remain                   |
| Columns renamed | All 18 to snake_case                                 |
| Columns added   | age_group, purchase_frequency_days, customer_segment |
| Columns removed | promo_code_used (duplicate)                          |
| Final shape     | 3,900 rows x 20 columns                              |

## 3. SQL Business Analysis

Ten business questions answered on the MySQL table shop (file: customer\_shopping\_behavior.sql). Each has a box for you to paste the query result.

```
USE customer;
```

### Q1. Revenue by gender

Total revenue generated by male vs female customers.

```

SELECT gender, SUM(purchase_amount) AS revenue
FROM shop
GROUP BY gender;

```

|   | gender | revenue |
|---|--------|---------|
| ► | Male   | 157890  |
|   | Female | 75191   |

### Q2. Discount users who spent above average

The subquery computes the overall average; the outer query keeps discounted purchases at or above it.

```

SELECT customer_id, purchase_amount FROM shop
WHERE discount_applied = "Yes"
  AND purchase_amount >= (SELECT AVG(purchase_amount) FROM shop)
ORDER BY purchase_amount DESC LIMIT 5;

```

|   | customer_id | purchase_amount |
|---|-------------|-----------------|
| ▶ | 1480        | 100             |
|   | 1413        | 100             |
|   | 96          | 100             |
|   | 1422        | 100             |
|   | 1457        | 100             |

### Q3. Top 5 products by average rating

Groups by product and ranks by rounded average review rating.

```
SELECT item_purchased, ROUND(AVG(review_rating),2) AS avg_review_rating
FROM shop
GROUP BY item_purchased
ORDER BY avg_review_rating DESC LIMIT 5;
```

|   | item_purchased | avg_review_rating |
|---|----------------|-------------------|
| ▶ | Gloves         | 3.86              |
|   | Sandals        | 3.84              |
|   | Boots          | 3.82              |
|   | Hat            | 3.8               |
|   | Handbag        | 3.78              |

### Q4. Standard vs Express shipping spend

Compares average purchase amount between the two shipping types.

```
SELECT shipping_type, AVG(purchase_amount)
FROM shop
WHERE shipping_type IN ("Standard","Express")
GROUP BY shipping_type;
```

|   | shipping_type | avg(purchase_amount) |
|---|---------------|----------------------|
| ▶ | Express       | 60.4752              |
|   | Standard      | 58.4602              |

### Q5. Do subscribers spend more?

Compares customer count, average spend and total revenue by subscription status.

```
SELECT subscription_status, COUNT(customer_id), AVG(purchase_amount) AS avg_spend,
      SUM(purchase_amount) AS revenue
FROM shop
GROUP BY subscription_status;
```

|   | count(customer_id) | avg_spend | revenue |
|---|--------------------|-----------|---------|
| ▶ | 1053               | 59.4919   | 62645   |
|   | 2847               | 59.8651   | 170436  |

## Q6. Top 5 products by discount rate

CASE WHEN turns each row into 1 or 0; the sum divided by the row count gives the percentage of discounted sales.

```
SELECT item_purchased,  
       ROUND(100.0 * SUM(CASE WHEN discount_applied='Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate  
FROM shop  
GROUP BY item_purchased  
ORDER BY discount_rate DESC LIMIT 5;
```

|   | item_purchased | discount_rate |
|---|----------------|---------------|
| ▶ | Hat            | 50.00         |
|   | Sneakers       | 49.66         |
|   | Coat           | 49.07         |
|   | Sweater        | 48.17         |
|   | Pants          | 47.37         |

## Q7. Customer segments

Buckets customers into New, Returning and Loyal by previous purchases and counts each.

```
with customer_type as (  
  SELECT customer_id, previous_purchases,  
  CASE  
    WHEN previous_purchases = 1 THEN 'New'  
    WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'  
    ELSE 'Loyal'  
  END AS customer_segment  
FROM customer)  
  
select customer_segment,count(*) AS "Number of Customers"  
from customer_type  
group by customer_segment;
```

|   | customer_segment | Number of Customers |
|---|------------------|---------------------|
| ▶ | Loyal            | 3116                |
|   | Returning        | 701                 |
|   | New              | 83                  |

## Q8. Top 3 products in each category

A CTE ranks products within each category using a window function; the outer query keeps ranks 1 to 3.

```
WITH item_counts AS (  
  SELECT category, item_purchased, COUNT(customer_id) AS total_orders,  
         ROW_NUMBER() OVER (PARTITION BY category  
                             ORDER BY COUNT(customer_id) DESC) AS item_rank  
  FROM shop  
  GROUP BY category, item_purchased)  
SELECT item_rank, category, item_purchased, total_orders  
FROM item_counts  
WHERE item_rank <= 3;
```

|   | item_rank | category    | item_purchased | total_orders |
|---|-----------|-------------|----------------|--------------|
| ▶ | 1         | Accessories | Jewelry        | 171          |
|   | 2         | Accessories | Sunglasses     | 161          |
|   | 3         | Accessories | Belt           | 161          |
|   | 1         | Clothing    | Blouse         | 171          |
|   | 2         | Clothing    | Pants          | 171          |
|   | 3         | Clothing    | Shirt          | 169          |
|   | 1         | Footwear    | Sandals        | 160          |
|   | 2         | Footwear    | Shoes          | 150          |
|   | 3         | Footwear    | Sneakers       | 145          |
|   | 1         | Outerwear   | Jacket         | 163          |
|   | 2         | Outerwear   | Coat           | 161          |

### Q9. Do repeat buyers subscribe?

Counts subscribers and non-subscribers among customers with more than 5 previous purchases.

```
SELECT subscription_status, COUNT(customer_id) AS repeat_buyers
FROM shop
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

|   | subscription_status | repeat_buyers |
|---|---------------------|---------------|
| ▶ | Yes                 | 958           |
|   | No                  | 2518          |

### Q10. Revenue by age group

Shows each age group's revenue contribution.

```
SELECT age_group, SUM(purchase_amount) AS revenue
FROM shop
GROUP BY age_group
ORDER BY revenue;
```

|   | age_group   | revenue |
|---|-------------|---------|
| ▶ | Senior      | 55763   |
|   | Adult       | 55978   |
|   | Middle-Aged | 59197   |
|   | Young Adult | 62143   |

## 4. Power BI Dashboard

The cleaned data was visualised in a one-page dashboard with KPI cards and category, gender, shipping, subscription and age-group views.



## 5. Key Insights

- **Clothing leads:** 1,737 sales and \$104.26K, about 45% of the \$233K revenue. Accessories follow with \$74.20K.
- **Male-heavy customer base:** 2,652 male vs 1,248 female customers, roughly 68% / 32%.
- **Few subscribers:** only 27% of customers subscribe, leaving 73% as a conversion opportunity.
- **Even age spend:** revenue ranges from \$55.76K (Senior) to \$62.14K (Young Adult).
- **Balanced shipping:** all six shipping types hold 16 to 17.5%, with Free Shipping highest at 17.49%.
- **Satisfaction:** average review of 3.75 out of 5 leaves room for improvement.

## Recommendations

- Promote subscriptions to the 73% non-subscribers, starting with repeat buyers (Q5 and Q9).
- Run targeted campaigns to grow the female customer segment.
- Bundle Outerwear and Footwear, the lowest-revenue categories, with Clothing best-sellers.
- Reward Loyal customers and nudge New ones toward a second purchase.

## 6. Skills Demonstrated

| Area     | Skills                                                              |
|----------|---------------------------------------------------------------------|
| Python   | Pandas, data cleaning, imputation, feature engineering, SQLAlchemy  |
| SQL      | Aggregations, subqueries, CASE, CTEs, window functions              |
| BI       | Power BI dashboard design, KPI cards, data storytelling             |
| Business | Customer segmentation, revenue analysis, actionable recommendations |